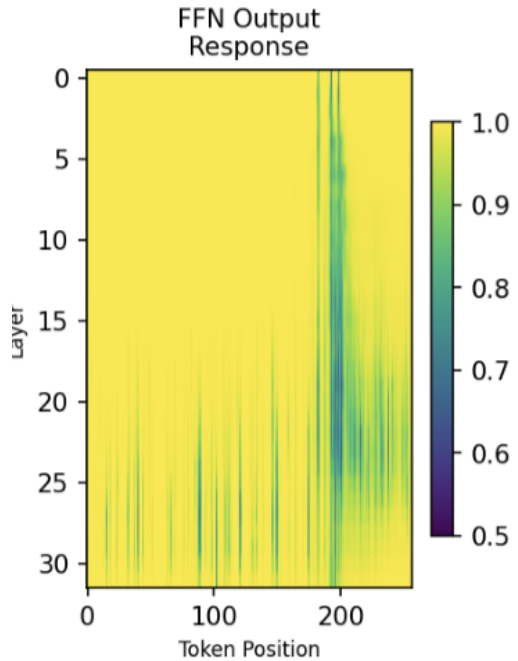
Goal: On top of the Fast-dLLM v2 codebase, implement two compute-skipping policies and report accuracy vs. FLOPs-reduction curves. (**Run 100 samples of GSM8K on 7B model**)

1. Token-level: Calculate the cosine similarity of token hidden states across adjacent denoising steps, and skip computing the corresponding tokens by reusing the previous step's outputs for that layer when the cosine similarity of hidden state input is sufficiently high. Do this on all layers of all tokens because the similarity difference may vary due to bidirectional attention and layer normalization.
    a. For per token, cosine similarity, 0.995, 0.99, 0.98, 0.97, 0.96
    b. For the top k% (tokens that changed the most): k = 25, 50.
    c. Token positions reused (or not reused) in the attention module are applied directly to the same positions in the MLP.
2. Layer-level: Calculate the cosine similarity of layer hidden states across adjacent denoising steps for the same layer, and skip computing the corresponding layers by reusing the previous step's layer outputs when the cosine similarity is sufficiently high.
    a. For cosine similarity, 0.999, 0.995, 0.99, 0.98, 0.97
    b. For similarity, compute the maximum or average per-token similarity as the criterion.
3. Instructions applied to all token-level and layer-level.
    a. In fastdllm-v2, they perform prefill and decode. For decoding, there is no skipping in the very first denoising step in each block. Token-level or layer-level sparsity starts from the second step.
    b. Log per step, per layer info regarding number of tokens are not reused (extra FLOPs) for each instances (will be used for figure plotting)
    c. Modify this and other related files.
    d. All similarities are computed on the hidden states after layer normalization.

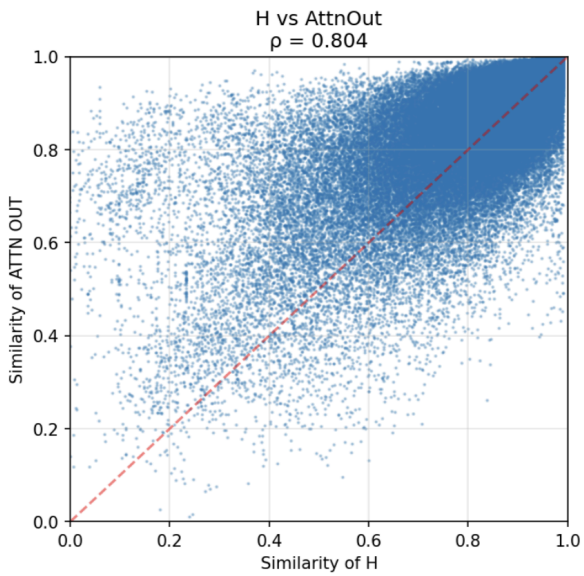
Deliverables

1. Baseline figure
    a. Layer skipping motivation plot (Total 3 figures)
        i. One sample
            1. x-axis: denoising step
            2. y-axis: layer
            3. heatmap: similarity
        ii. 100l samples of GSM8K (2 plots, mean and variance)
            1. x-axis: denoising step
            2. y-axis: layer
            3. heatmap: similarity
            4. For different steps, normalize that
    b. Token skipping motivation plot (per-token similarity) (Total 4 figures)
        i. One sample
            1. x-axis: token position
            2. y-axis: layer
            3. heatmap: similarity (average over denoising step)
            4. See the sample below



ii. 100 samples of GSM8K (2 plots, mean and variance)
    1. x-axis: denoising step
    2. y-axis: layer
    3. heatmap: similarity
    4. For different steps, normalize that

iii. Similarity hidden state versus similarity of attention output and FFN output covariance



c. Column skipping motivation plot (Total 3 figures)
    i. Attention similarity (also averaging all samples)
        1. x-axis: denoising step
        2. y-axis: layer

3. Heatmap: attention score similarity compared to last denoising step

ii. Attention score range (Logging 100 samples, all steps, all layers, all tokens of GSM8K)

1. x-axis: value range (log scale)
   a. 0.0001 - 0.001, 0.001 - 0.01, 0.01 - 0.1, 0.1 - 1
   b. Each with 10 ticks, for a total of 40.
2. y-axis: occurrence
3. Heatmap
4. Use the following to get the attn_weight

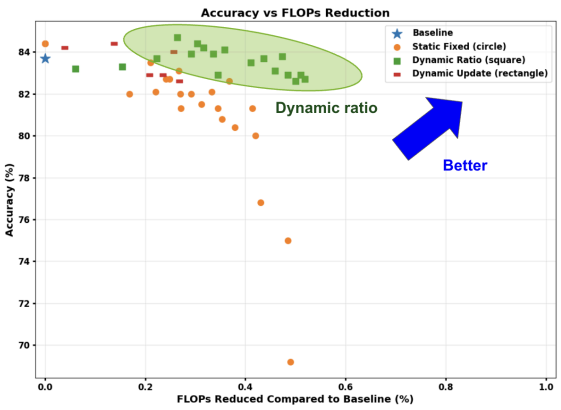
```
def _compute_attention_stats(self, query_states, key_states, value_states
    # query_states: [B, H_q, Q_Len, Head_Dim]
    # key_states: [B, H_k, K_Len, Head_Dim]
    # value_states: [B, H_k, K_Len, Head_Dim]

    # Handle GQA (Grouped Query Attention) if needed
    if query_states.size(1) != key_states.size(1):
        key_states = repeat_kv(key_states, self.num_key_value_groups)
        value_states = repeat_kv(value_states, self.num_key_value_groups)

    orig_dtype = query_states.dtype
    attn_weights = torch.matmul(query_states, key_states.transpose(2, 3))
    attn_weights = torch.exp(attn_weights.float() * self.scaling)
```

iii. FFN

1. x-axis: denoising step
2. y-axis: layer
3. Heatmap: similarity of temp (see below) compared to last denoising step

```
temp = self.act_fn(self.gate_proj(x)) * self.up_proj(x)
```

2. Acc vs FLOPs reduction figure
   a. Total of 18 settings (baseline (1), cosine similarity (5), topk (2), layer-level avg (5), layer-level max (5)) on one figure
   b. Average denoising steps over samples for each setting, use a table to show that.
   c. See the following style



3. Codes and instructions to run.
4. Write up regarding (a) how you implemented, (b) how you calculated the FLOPs reduction, (c) your observation, (d) propose an idea to further improve it.

Total 11 figures, 1 table and codes, 1 Writeup